ENGINEERING BUILD PLAN

RajyaRank Student Mobile App

A native Flutter app for iOS and Android, built on top of the existing RajyaRank API and web platform — full feature parity with the student web experience.

PREPARED	PLATFORM	STATUS
August 2026	**Flutter (iOS + Android)**	**Foundation build in progress**

1 Executive Summary

RajyaRank already has a full student experience on the web — courses, tests, previous-year papers, doubts, payments — served by a NestJS API that a mobile app can plug straight into. This is not a rebuild; it is a native port, backed by a backend that turns out to already be closer to mobile-ready than most.

The scope for the first release is **full parity with the web app**: everything a student can do on rajyarank.com today, plus push notifications (which the web app only partially has via browser push). The one thing genuinely out of scope for this release is offline lesson/PDF downloads — that doesn't exist on web either, and is real new product surface rather than a straight port.

KEY FINDING

The API's authentication layer already accepts mobile-style Bearer tokens ahead of browser cookies, and the permission engine that gates every endpoint has no knowledge of "web" vs "mobile" at all. Razorpay's existing order-create → checkout → verify contract maps directly onto the official Flutter payment plugin. Every lesson video, PDF, and previous-year paper is already served as a plain signed URL with no browser dependency. The gaps that do exist are small, specific, and listed below — not architectural rework.

2 What Ships in the First Release

Account & Auth

OTP login, email/password login, signup, Google sign-in, password reset, onboarding wizard, session management ("log out of all devices").

Learning

Dashboard, enrolled courses & curriculum, lesson player (video / PDF / YouTube embed), bookmarks, study plan, revision hub.

Tests

Test catalogue, timed attempt runner with autosave, submission, results, and a detailed per-question review.

Payments

Course & plan pricing, Razorpay checkout, coupon codes, saved cards, order history and receipts.

Account & Institutions

Profile, study goals, institution join/leave via access code, password management.

Engagement

Previous Year Question papers, Doubts (ask + teacher replies), Current Affairs, search, wishlist, support tickets.

INCLUDED THIS TIME, UNLIKE A TYPICAL V1

Push notifications ship in the first release rather than being deferred — the backend addition needed for it (device-token registration + a sender) reuses a fan-out point that already centralizes every notification-worthy event in the system, so the incremental cost is low.

DEFERRED TO A LATER RELEASE

Offline lesson/PDF downloads (new scope — no offline API exists yet, even on web); real-time push for doubt replies (the web app is poll-based today too); a companion tutor/staff app; page-level PDF progress tracking.

3 Technical Approach

CONCERN	CHOICE	WHY
State management	`flutter_riverpod`	Fits an app that's mostly async API-backed screens; testable, minimal boilerplate.
Navigation	`go_router`	Official package; needed for deep links from push notifications and shared links.
HTTP client	`dio` + interceptors	Attaches the auth header automatically, retries once through a token refresh on a 401.
Token storage	`flutter_secure_storage`	iOS Keychain / Android Keystore — standard for holding session tokens on-device.
Video	`video_player` / `chewie`	Matches the direct video files the web app already streams for most lessons.
PDF viewer	`pdfx`	Open-source; the web app has no real PDF viewer to port from (bare browser iframe), so this is a place mobile improves on web.
Payments	`razorpay_flutter`	Official plugin; matches the API's existing order-create → verify contract with no backend change to the core purchase flow.
Push	`firebase_messaging`	Covers both Android (FCM) and iOS (via Firebase/APNs) from one integration.
Visual design	RajyaRank's existing colour system	Navy, orange and teal tokens ported directly from the web app's design system — the mobile app looks like the same product from day one, no separate design phase.
CI / CD	GitHub Actions + Fastlane	Automated builds to Google Play internal testing and Apple TestFlight, consistent with how the API and worker already deploy.

4 Backend Changes Required

All additive — nothing the web app does today changes. Four small, scoped changes unlock the mobile app; two are already shipped.

#	CHANGE	STATUS	WHY IT'S NEEDED
1	Return session tokens in the login/signup response body, not only as browser cookies	SHIPPED	The web app keeps using cookies unchanged; the mobile app stores these tokens itself since it has no cookie jar.
2	Accept a refresh token in the request body as a fallback to the cookie	SHIPPED	Lets the mobile app silently renew its session the same way the web app's background refresh already works.
3	Native Google Sign-In token exchange	PLANNED	The web app's Google login is a browser redirect flow; a native app needs a direct token-exchange endpoint instead.
4	Push notification device registration + sender	PLANNED	The web app only has browser push today; mobile needs the equivalent for FCM/APNs, wired into the existing notification system.

5 Roadmap

M0 IN PROGRESS

Foundation

Project setup, design system, navigation shell, secure auth, first backend changes. Apple Developer Program enrollment starts in parallel — it can take days to verify and shouldn't be left until late.

M1

Auth & Onboarding

OTP, email/password, Google sign-in, password reset, the onboarding wizard, and device/session management.

M2

Core Learning

Dashboard, courses, the lesson player across video/PDF/embedded formats, study plan, revision.

M3

Tests

Catalogue, the timed attempt runner, submission, results, and detailed review.

M4

Payments & Account

Course/plan purchase via Razorpay, saved cards, order history, profile and institution management.

M5

Engagement

Previous Year Questions, Doubts, Current Affairs, search, wishlist, support.

M6

Notifications

In-app notification centre and push notifications, with deep links into content.

M7

Polish & Submission

Accessibility and localization QA, crash reporting, performance pass, store listings, TestFlight/Play internal testing, submission.

6 Current Status

AVAILABLE NOW

A working Flutter foundation build is running on Android — real sign-in against the live RajyaRank API, and a live dashboard showing real account data (study streak, course progress, today's plan). Distributed as an installable `.apk` for direct device testing, ahead of any app-store listing.

IOS IS BLOCKED ON ONE EXTERNAL STEP

Building an installable iOS app requires Apple's own toolchain (Xcode on macOS), which only runs through cloud build infrastructure with real Apple Developer Program credentials attached. An Apple Developer Program enrollment ($99/year, with identity verification that can take several days) needs to happen before any iOS build — including a simple on-device test install — can be produced. This is the single external dependency blocking iOS progress; everything else is engineering work already underway.

7 Next Steps

1. Continue M1–M7 in sequence, each ending in a testable Android build.
2. Start Apple Developer Program enrollment now, in parallel — it gates iOS builds and TestFlight distribution regardless of when engineering gets there.
3. Wire up GitHub Actions + Fastlane once the first few milestones are stable, for automatic Play/TestFlight builds instead of manual ones.